

ResearchPilot AI

Complete Technical Documentation

Project: ResearchPilot AI

Type: MCA Final Year Project

Covers: NLP · GenAI · AI/ML · Cloud Analytics · Frontend · Backend · Database

Date: September 2026

NLP · Generative AI · AI/ML · Cloud Analytics & Big Data

Table of Contents

- [Project Overview](#1-project-overview)
- [System Architecture](#2-system-architecture)
- [Tech Stack](#3-tech-stack)
- [Frontend](#4-frontend)
- [Backend](#5-backend)
- [Database](#6-database)
- [NLP Module](#7-nlp-module)
- [Generative AI Module](#8-generative-ai-module)
- [AI/ML Module](#9-aiml-module)
- [Cloud Analytics & Big Data Module](#10-cloud-analytics--big-data-module)
- [Application Workflow](#11-application-workflow)
- [Each Module — What It Does](#12-each-module--what-it-does)
- [API Endpoints Reference](#13-api-endpoints-reference)

1. Project Overview

ResearchPilot AI is a full-stack, AI-powered academic research management platform.

Researchers upload PDF papers which are automatically analysed, indexed, and made searchable. Users can then:

- Semantically search across all their uploaded papers
- Chat with an AI assistant grounded strictly in uploaded evidence
- Detect research gaps in their library
- Generate a research roadmap from those gaps
- Compare papers side-by-side
- Format citations in APA, IEEE, or MLA
- View topic clusters and keyword trends across their library

The platform is designed so every AI answer is traceable back to real, uploaded evidence — no hallucinated facts or invented citations.

2. System Architecture



[illegible]

- Frontend runs on **port 3000** (Vite dev) or served as static files (production)
- Backend runs on **port 8000** (Uvicorn/FastAPI)
- All communication is REST over HTTP — no WebSockets
- The FAISS index lives in memory and is rebuilt from the database on first use
- PDFs are stored as files on disk; only metadata goes to the database

3. Tech Stack

Layer	Technology	Version
Frontend framework	React	18+
Language (frontend)	TypeScript	~5.9
Build tool	Vite	5+
CSS framework	Tailwind CSS	4
UI components	Radix UI primitives	Various
Routing	Wouter	^3.3.5
Authentication	Clerk	^6.15
Server state	TanStack React Query	Latest
Animation	Framer Motion	Latest
Charts	Recharts	^2.15
Backend framework	FastAPI	0.115.6
ASGI server	Uvicorn	(via FastAPI)

Layer	Technology	Version
Language (backend)	Python	3.11
ORM	SQLAlchemy	2.0.36
Database	SQLite	(file-based)
PDF processing	PyMuPDF (fitz)	1.25.3
NLP	scikit-learn TF-IDF	1.6.1
NLP (optional)	spaCy	3.8.4
Embedding model	sentence-transformers	3.4.1
Vector search	FAISS CPU	1.9.0
LLM provider	Groq (LLaMA-3.3-70B)	(via groq SDK)
Big Data analytics	pandas	2.2.3
ML clustering	scikit-learn KMeans	1.6.1
Containerisation	Docker	python:3.11-slim
Frontend deploy	Vercel	—
Backend deploy	Render	—

4. Frontend

4.1 Overview

The entire frontend is a React single-page application (SPA) built in TypeScript. All pages are defined in `artifacts/researchpilot/src/App.tsx`. Routing uses Wouter — a lightweight alternative to React Router. There are no separate page files; everything is inline in one large component tree.

4.2 Authentication

Two modes:

Local / Dev mode (no Clerk key): A simple `LocalStorage`-based login that stores the user's name and email. No real password validation. Used for local development.

Production mode (Clerk key present): Full Clerk authentication with `ClerkProvider`, `<Show when="signed-in">` guards on every protected route, and automatic token/session management. A `ClerkQueryClientCacheInvalidator` clears the React Query cache on user switch to prevent data leakage between users.

User identity is passed to the backend as an `X-User-Email` HTTP header derived from the logged-in user's email address. This scopes all database queries to that user's papers.

4.3 Pages and Routes

`/` — Home Landing

The public marketing page. Shown only to unauthenticated users. Contains a hero section, feature highlights, and links to sign in or create an account. Redirects signed-in users to `/dashboard`.

`/sign-in` and `/sign-up`

Authentication pages. In production they render Clerk's `<SignIn>` and `<SignUp>` components with a custom brand appearance. In dev mode they render custom local forms.

`/dashboard` — Research Overview

The first page after login. Displays:

- **4 stat cards:** papers in library, papers read this month, evidence threads, saved citations
- **Continue your thread:** cards for papers currently marked "Reading" with a progress bar
- **Intelligence note:** a surfaced research gap from the library
- **Most cited in your library:** top 3 papers by citation count

Acts as the user's research pulse view — a quick summary of what needs attention next.

`/library` — Evidence Library

Full paper management. Features:

- **Search bar:** client-side filter on title, authors, and tags
- **Status filter:** All papers / Favorites / Reading / Unread / Complete
- **Sort:** Recent (by year) or Most cited
- **Import paper button:** triggers a file input `<input type="file" accept=".pdf">` → calls `POST /api/papers` to upload and process the PDF
- **Paper rows:** color-accented cards with status badge, venue, year, authors, tags, favorite toggle, and a "open reader" button

Every paper upload triggers the full NLP + embedding pipeline on the backend.

`/reader` — Paper Reader

Split-layout reading experience. Contains:

- **PDF iframe:** embeds the uploaded PDF via `GET /api/papers/{id}/file`
- **Paper switcher:** dropdown to switch between papers without leaving the reader
- **PaperOrganizer:** inline form to edit collection name and tags (saved via

`PATCH /api/papers/{id}`)

- **ReaderContent tabs:**

- **Evidence tab:** fetches semantic search results for

`"findings evidence results conclusions"` scoped to the current paper, displays as numbered evidence cards with relevance percentages

- **Abstract tab:** shows the auto-extracted abstract proxy
 - **Notes tab:** free-text textarea saved back to the backend

`/intelligence` — Research Intelligence

The AI-powered analysis page. Three sub-tabs:

Gaps tab: Shows detected research gaps — limitations, future work suggestions, and methodological weaknesses found by running semantic queries against the paper library. Each gap card shows a title, confidence score (semantic similarity %), evidence excerpt, and the source paper.

Compare tab: Side-by-side comparison of two papers extracted from the library. Shows year, top keywords, shared keywords (overlap detection), abstract excerpts, and sentence count. Built from `GET /api/intelligence/comparison`.

Roadmap tab: A 3-step research roadmap automatically generated from detected gaps. Each step has a phase label (next / then / later), a research objective, a methodology description, and measurable success metrics.

`/citations` — Citation Manager

Client-side citation formatting. Users:

- Select which papers to cite using checkboxes
- Pick a citation style: APA 7, IEEE, or MLA 9
- See live-formatted citations updated instantly as they change style
- Copy a single citation or copy the full bibliography

All citation formatting happens in the browser using the `formatCitation()` function —

no server round-trip needed. The function handles author name inversion, volume/issue/pages/article-number, and DOI links following each style's exact formatting rules.

`/chat` — Ask Your Library

A conversational chat interface. The user types a question and the backend:

- Searches the FAISS index for relevant evidence chunks
- Passes them to LLaMA-3.3-70B with a strict evidence-only prompt
- Returns an answer + source list

The UI shows the user's message on the right, the AI response on the left, with source papers listed below each answer including relevance scores. Quick-prompt buttons provide one-click research questions like "What gaps are emerging?" or "Compare the methods".

`/settings` — Workspace Settings

Persisted to `localStorage`. Allows the user to configure:

- Research area (display label)
- Default citation style (APA / IEEE / MLA)
- Focus mode on reader open (hides secondary panels)
- Compact library rows
- Show evidence highlights in reader
- Include source evidence in AI answers
- Show similarity score percentages
- Default number of semantic search results (5 / 10 / 20)

4.4 Shell (Persistent Layout)

The `Shell` component wraps all authenticated pages. It provides:

- **Sidebar navigation** with links to all 7 pages and a backend status indicator (green dot = online, red dot = offline). The indicator polls `GET /healthz` every 15 seconds.
- **Global search bar** in the header that runs `GET /api/search` across all papers, shows results in a dropdown with similarity percentages, and clicking a result opens that paper in the Reader.
- **User avatar** in the top-right corner with initials, links to Settings.
- **Mobile hamburger menu** — the sidebar collapses on small screens.

4.5 Key Frontend Utility Functions

`paperFromApi(paper)` — Converts the backend API response into the frontend `Paper` type. It performs client-side parsing of PDF metadata: extracts the real paper title from

the PDF's built-in `title` field, parses authors from comma/semicolon-delimited strings, extracts the publication year from the `subject` field using regex, parses journal name, volume, issue, pages, article number, and DOI. This means papers show correct bibliographic information even though the backend only stores raw metadata JSON.

`formatCitation(paper, style)` — Pure function for APA 7, IEEE, and MLA 9 citation formatting. Handles multi-author formatting rules per style (ampersand vs "and"), name inversion, article number vs page number priority, and DOI URL formatting.

`apiHeaders()` — Returns the `X-User-Email` header from `localStorage`. This scopes all API calls to the logged-in user's papers.

4.6 Frontend Package Summary

Key npm packages used:

Package	Purpose
<code>@clerk/react</code>	Cloud authentication with sign-in/sign-up UI
<code>@tanstack/react-query</code>	Async server state caching
<code>wouter</code>	SPA routing
<code>lucide-react</code>	Icon library (80+ icons used)
<code>tailwindcss</code>	Utility CSS classes
<code>@radix-ui/*</code> (30 packages)	Accessible headless UI primitives
<code>recharts</code>	Charts (available but analytics pages currently use raw data)
<code>framer-motion</code>	Page transition animations
<code>react-hook-form</code> + <code>zod</code>	Form state + validation
<code>cmdk</code>	Command palette
<code>sonner</code>	Toast notifications
<code>vaul</code>	Drawer/sheet component

5. Backend

5.1 Overview

The backend is a Python FastAPI application (`backend/app/main.py`) served by Uvicorn. It exposes a REST API consumed by the React frontend. It wires together all AI/ML modules and manages the SQLite database via SQLAlchemy 2.0.

5.2 API Routes

Method	Path	Purpose
GET	<code>/healthz</code>	Health check
GET	<code>/api/papers</code>	List all papers for the user
POST	<code>/api/papers</code>	Upload a PDF — triggers full NLP + embedding pipeline
GET	<code>/api/papers/{id}/file</code>	Serve a PDF file inline
PATCH	<code>/api/papers/{id}</code>	Update paper metadata (title, collection, tags, notes, favorite)
GET	<code>/api/search</code>	Semantic search across the FAISS index
POST	<code>/api/chat</code>	RAG-based chat question answering
GET	<code>/api/intelligence/gaps</code>	Detect research gaps in the library
GET	<code>/api/intelligence/roadmap</code>	Generate a research roadmap from gaps
GET	<code>/api/intelligence/comparison</code>	Compare papers side-by-side
POST	<code>/api/index/rebuild</code>	Rebuild the FAISS index from all stored papers
GET	<code>/api/analytics/clusters</code>	K-Means topic clustering of papers
GET	<code>/api/analytics/trends</code>	Keyword frequency and year trend analysis

5.3 Request/Response Models (Pydantic)

`PaperResponse` — returned from paper endpoints:

id, title, filename, abstract, keywords (list), sentence_count, source_metadata (dict), created_at, collection, tags (list), notes, favorite (bool).

`PaperUpdate` — accepted by PATCH:

title, collection, tags (list), notes, favorite — all optional.

`SemanticSearchResult` — returned from `/api/search`:

paper_id, title, text (chunk), chunk_index, score.

`ChatRequest` — sent to `/api/chat`:

question (required), top_k (default 5), paper_id (optional, scopes to one paper).

`ChatResponse` — returned from `/api/chat`:

question, answer, sources (list with paper_id, paper_title, chunk_id, similarity_score, evidence text), status (generated / fallback / no_evidence), model name.

5.4 Schema Migration

There is no Alembic or migration framework. Instead, `main.py` runs on every startup:

- `Base.metadata.create_all(engine)` — creates the `papers` table if it doesn't exist
- `PRAGMA table_info(papers)` — reads the current column list
- `ALTER TABLE papers ADD COLUMN ...` — adds any missing columns

This is an additive-only migration pattern. It handles incremental column additions but cannot modify or remove existing columns.

5.5 CORS

Configured with `allow_origins=["*"]` — all origins are permitted. Appropriate for development and single-user deployments. For multi-tenant production use, origins should be restricted to the frontend domain.

5.6 PDF Storage

Uploaded PDFs are stored in `backend/storage/{paper_id}.pdf`. The directory is created automatically on startup with `UPLOAD_DIR.mkdir(parents=True, exist_ok=True)`. The UUID is generated with `uuid4()` at upload time. Only the metadata and extracted text are stored in the database; the raw PDF bytes live on disk.

5.7 Python Package Summary

Package	Version	Purpose
<code>fastapi</code>	0.115.6	Web framework + OpenAPI auto-docs
<code>python-dotenv</code>	1.0.1	<code>.env</code> file loading
<code>python-multipart</code>	0.0.20	Multipart form / file upload parsing
<code>SQLAlchemy</code>	2.0.36	ORM + SQL toolkit
<code>PyMuPDF</code>	1.25.3	PDF text and metadata extraction
<code>spacy</code>	3.8.4	Optional NLP sentencizer (off by default)
<code>scikit-learn</code>	1.6.1	TF-IDF, KMeans, silhouette score
<code>pandas</code>	2.2.3	Data analysis and trend aggregation
<code>sentence-transformers</code>	3.4.1	Sentence embedding model
<code>faiss-cpu</code>	1.9.0	Vector similarity search index
<code>pydantic-settings</code>	2.7.1	Settings management
<code>pytest</code>	8.3.4	Testing framework
<code>httpx</code>	0.28.1	Async HTTP client (used by Groq SDK)
<code>groq</code>	(unlisted)	Groq API SDK for LLaMA inference

6. Database

6.1 Engine and ORM

- **Database engine:** SQLite (file: `backend/researchpilot.db`)
- **ORM:** SQLAlchemy 2.0 with `DeclarativeBase` and `Mapped` typed columns
- **Session style:** with `Session(engine)` as `session:` context manager per request
- **Migrations:** Manual `PRAGMA table_info` + `ALTER TABLE` on startup

6.2 Table: ``papers``

This is the only table in the database. All application state lives here.

Column	Type	Default	Description
<code>id</code>	VARCHAR(36) PK	—	UUID v4 generated at upload
<code>title</code>	VARCHAR(500) NOT NULL	—	Filename-derived initially; frontend overrides from PDF metadata
<code>filename</code>	VARCHAR(500) NOT NULL	—	Original uploaded filename
<code>abstract</code>	TEXT	""	First 3 sentences of extracted text (max 1,200 chars)
<code>extracted_text</code>	TEXT	""	Full raw text from PyMuPDF (all pages)
<code>keywords</code>	TEXT	""	Comma-separated TF-IDF keywords (up to 8)
<code>sentence_count</code>	INTEGER	0	Total sentence count from NLP segmentation
<code>source_metadata</code>	TEXT	"{}"	JSON blob of PDF built-in metadata (title, author, subject, createDate, etc.)
<code>owner_email</code>	VARCHAR(320)	"legacy"	User identity from X-User-Email header
<code>collection</code>	VARCHAR(200)	"My library"	User-defined collection grouping
<code>tags</code>	TEXT	""	Comma-separated user-defined tags
<code>notes</code>	TEXT	""	Free-text research notes
<code>favorite</code>	INTEGER	0	Boolean stored as 0/1
<code>created_at</code>	DATETIME	<code>datetime.now(UTC)</code>	Upload timestamp

6.3 Serialisation Patterns

SQLite has no native array or JSON types. The following conventions are used:

Field	Storage format	Deserialised to
keywords	"ai,machine learning,nlp"	list[str] in to_response()
tags	"review,important"	list[str] in to_response()
source_metadata	'{"title":"...", "author":"..."}'	dict[str,str] via json.loads()
favorite	0 or 1	bool — cast in to_response()

6.4 File Storage

PDFs are NOT stored in the database. They are saved as files at:

```
backend/storage/{paper_id}.pdf
```

The `storage/` directory is created automatically on startup. The `/api/papers/{id}/file` endpoint serves them via FastAPI's `FileResponse` with `media_type="application/pdf"`.

7. NLP Module

File: `backend/app/analysis.py`

7.1 What It Does

Processes every uploaded PDF to extract raw text, identify key terms, and generate a short abstract proxy. This runs as part of the upload pipeline before the paper is stored in the database.

7.2 Pipeline Steps

```
PDF bytes (upload) ↓ extract_pdf() — PyMuPDF opens the PDF stream, concatenates all page text ↓
and extracts the metadata dictionary
normalize_text() — collapses all whitespace runs to single
spaces ↓ segment_sentences() — splits text into sentence list ↓ — either regex (default) or
spaCy sentencizer (if ENABLE_SPACY=1)
abstract proxy — first 3 sentences, capped at 1,200
characters ↓ analyze_text() — runs TF-IDF on the full normalised text ↓ keywords — top 8 terms
(unigrams + bigrams) by TF-IDF score
```

7.3 Algorithms Used

TF-IDF (Term Frequency — Inverse Document Frequency)

TF-IDF measures how important a word is to a document relative to a corpus. Here it is applied to a single document (the paper itself) using `TfidfVectorizer`:

- `stop_words="english"` — removes common English words (the, is, of, etc.)
- `max_features=12` — vocabulary limited to 12 most frequent terms
- `gram_range=(1, 2)` — includes both single words and two-word phrases
- Top 8 terms by descending score are returned as keywords

Sentence Segmentation

Default mode uses a simple regex split on sentence-ending punctuation followed by whitespace: `re.split(r"(?<=[.!?])\s+", text)`. This covers most academic text reliably.

If `ENABLE_SPACY=1` is set in the environment, spaCy's `sentencizer` pipeline is used instead for more accurate boundary detection in complex academic prose.

7.4 Functions Reference

Function	Input	Output
<code>extract_pdf(content)</code>	PDF bytes	<code>(text: str, metadata: dict)</code>
<code>extract_pdf_text(content)</code>	PDF bytes	<code>text: str</code>
<code>normalize_text(text)</code>	Raw text	Cleaned text (collapsed whitespace)
<code>segment_sentences(text)</code>	Normalised text	<code>list[str]</code> of sentences
<code>analyze_text(text)</code>	Normalised text	<code>{abstract, keywords, sentence_count, character_count}</code>

8. Generative AI Module

File: `backend/app/gemini.py`
Used by: `backend/app/rag.py`

8.1 What It Does

Provides the large language model (LLM) generation capability. Despite the file and class

being named `GeminiService`, it actually calls the **Groq API** with **LLaMA-3.3-70B-Versatile** (the naming is a legacy artifact from an earlier version that used Google Gemini).

Groq is a hardware-accelerated LLM inference provider that offers extremely fast token generation using custom Language Processing Units (LPUs).

8.2 Model

Property	Value
Provider	Groq API
Model	LLaMA 3.3 70B Versatile
Temperature	0.7
Max tokens	1,024 per response
Context	Passed via user prompt

8.3 How It Works

- On initialisation, `GeminiService` reads `GROQ_API_KEY` and `GROQ_MODEL` from environment variables. If no API key is found, `available` returns `False`.
- The `generate(prompt)` async method calls `client.chat.completions.create()` using a single "user" role message.
- On any exception (network error, rate limit, invalid key), it logs the error and returns `None` — this triggers a graceful fallback in `RAGService`.
- When `available` is `False` or generation returns `None`, the RAG system returns the retrieved evidence text directly without AI synthesis, still labelling the response status as "fallback".

8.4 RAG Pipeline (rag.py)

The `RAGService` in `rag.py` orchestrates the full Retrieval-Augmented Generation flow:

```
User question ↓ SemanticIndex.search(question, top_k) ↓ returns evidence chunks with similarity scores
Filter chunks where score ≥ 0.2 ↓ build_evidence_prompt(question, evidence) ↓ strict prompt: answer ONLY from evidence, no invented facts
GeminiService.generate(prompt) → Groq API → LLaMA 3.3 70B ↓ Return {answer, sources, status, model}
```

Evidence prompt design:

The prompt explicitly instructs the LLM:

- Answer using ONLY the supplied research evidence
- If evidence is insufficient, say so explicitly
- Do not invent facts
- Do not fabricate citations
- Do not claim information not in the retrieved evidence

Each evidence chunk is formatted with paper ID, title, chunk index, and similarity score so the LLM can attribute statements precisely.

Status values:

- "generated" — LLM produced an answer
- "fallback" — LLM unavailable; evidence returned without synthesis
- "no_evidence" — no chunks scored above the minimum threshold

9. AI/ML Module

9.1 Semantic Embedding and Search

File: `backend/app/semantic.py`

Model: all-MiniLM-L6-v2

A Sentence Transformer model from HuggingFace that converts sentences/paragraphs into 384-dimensional dense vectors. It is a distilled, fine-tuned version of BERT trained specifically for semantic similarity tasks. It is lightweight (~80 MB) and runs fast on CPU.

FAISS Index

FAISS (Facebook AI Similarity Search) is a library for efficient similarity search over dense vectors. This project uses `IndexFlatIP`:

- **Flat** = brute-force exhaustive search (no approximation)
- **IP** = inner-product similarity
- Cosine similarity is achieved by L2-normalising all vectors before adding them to the

index (via `normalize_embeddings=True` in `sentence-transformers`)

```
Paper text ↓ chunk_text() ↓ sentence_transformers.encode() normalize_embeddings=True ↓ float32  
vectors [384-dim each] ↓ faiss.IndexFlatIP.add(vectors) ↓ EvidenceChunk list (paper_id, title,  
text, chunk_index)
```

On search:

```
Query string ↓ sentence_transformers encode([query], normalize_embeddings=True) ↓  
faiss.IndexFlatIP.search(vector, k) ↓ Returns (scores, indices) → mapped back to EvidenceChunk  
list ↓ Optionally filtered by paper_id
```

Text Chunking Pipeline

`chunk_text()` in `semantic.py` is one of the most important functions in the system.
It determines what content actually gets indexed.

Steps:

- Split text into lines, scan for section headers ("references", "bibliography", "acknowledgements" etc.) — mark everything after as reference/boilerplate sections and skip it
- Join remaining body lines and split into sentences on .
- Filter out sentences shorter than 40 characters
- Filter out sentences matching any of 40+ boilerplate markers (publisher names, copyright notices, funding statements, DOI lines, ISSN, received/accepted dates, etc.)
- Group into sliding windows of 6 sentences per chunk
- Drop chunks shorter than 200 characters

This ensures the FAISS index contains only genuine research content — methods, results, discussions, conclusions — not publisher boilerplate.

9.2 Research Gap Detection

File: `backend/app/intelligence.py` — `detect_gaps()`

Three semantic queries are run against the FAISS index to find evidence chunks that describe limitations, future work, and methodological gaps:

Gap type	Semantic query
limitations	"Limitations and constraints described in the research evidence."
future_work	"Future work and unresolved research problems described in the evidence."
method_gap	"Missing methods, weak evaluation, or underexplored comparisons in the evidence."

For each query, chunks are filtered by:

- Similarity score ≥ 0.3

- Text length ≥ 150 characters
- At least 2 full sentences (2+ occurrences of `.`)
- Not already returned for a different gap type (deduplication by `paper_id` + `chunk_index`)

Results are sorted by similarity score descending — the most relevant evidence surfaces first.

9.3 Roadmap Generation

File: `backend/app/intelligence.py` — `build_roadmap()`

Each detected gap is converted to a research roadmap step using a template system:

Gap type	Phase	Template objective
<code>limitations</code>	next	Re-test results under the constraints the source acknowledges
<code>future_work</code>	then	Advance the proposed direction into a concrete study design
<code>method_gap</code>	later	Introduce a stronger evaluation method and benchmark it

The `_topic_hint()` function extracts a short phrase (4 meaningful words after stop-word removal) from the gap evidence to personalise the objective text.

10. Cloud Analytics & Big Data Module

File: `backend/app/analytics.py`

10.1 What It Does

Provides two analytics functions over the paper library:

- **Topic clustering** — groups papers by research theme using unsupervised ML
- **Research trends** — tracks keyword frequency and ingestion volume over time

In a big data context, this module represents the **analysis layer** of a data pipeline: it aggregates and processes a corpus of research documents to surface patterns that are not visible when reading individual papers.

10.2 K-Means Topic Clustering

Function: `cluster_papers(papers)`

Algorithm:

- For each paper, build a document string: `title + abstract + keywords`
- Vectorise all documents using TF-IDF:
 - `stop_words="english"` — removes common English words
 - `max_features=200` — limits vocabulary to 200 most informative terms
 - Output: a sparse TF-IDF matrix of shape `(n_papers, 200)`
- Determine cluster count `k` using the square-root heuristic:

`k = min(max(2, round(sqrt(n))), n - 1)`

This is a standard rule of thumb that balances granularity with stability.

- Run `KMeans(n_clusters=k, random_state=42, n_init=10)`:
 - `n_init=10` runs K-Means 10 times with different initialisations and keeps the best
 - `random_state=42` ensures reproducible results
- Compute **Silhouette Score**: measures clustering quality as the mean ratio of intra-cluster cohesion to inter-cluster separation. Range: [-1, 1]; higher = better.
- Return cluster assignments per paper, cluster sizes, and the silhouette score.

Use case in big data: As the paper library grows (hundreds of papers), this enables automatic discovery of research sub-topics without manual tagging — equivalent to unsupervised topic discovery on a large document corpus.

Requires minimum 2 papers. Returns `"insufficient_data"` status for a single paper.

10.3 Research Trend Analysis

Function: `research_trends(papers)`

Uses **pandas** for aggregated data analysis over the paper collection:

- Builds a DataFrame with columns: `paper_id`, `title`, `ingestion_year`, `keywords`
- Groups by `ingestion_year` using `groupby("ingestion_year").size()` to get papers-per-year counts
- Uses `collections.Counter` to count keyword occurrences across all papers, returning the top 20 most frequent keywords
- Returns `{paper_count, ingestion_years: [{year, paper_count}], keyword_frequency: [{keyword, paper_count}]}`

Big data relevance: This trend view is designed to scale — as more papers are uploaded over months and years, the ingestion timeline and keyword frequency charts reveal how the research focus of a library shifts over time.

10.4 Analytics API Endpoints

Endpoint	Function	Returns
GET <code>/api/analytics/clusters</code>	<code>cluster_papers()</code>	Cluster assignments, sizes, silhouette score
GET <code>/api/analytics/trends</code>	<code>research_trends()</code>	Paper counts per year, top 20 keywords

11. Application Workflow

11.1 PDF Upload Workflow

User selects a PDF in the Library page ↓ Frontend: POST `/api/papers` (multipart form-data) ↓ Backend validates MIME type (must be application/pdf) ↓ Generates `paper_id = uuid4()` ↓ Saves bytes to `/storage/{paper_id}.pdf` ↓ `analysis.py: extract_pdf()` → PyMuPDF opens PDF stream → Concatenates all page text → Extracts metadata dict (title, author, subject, dates) ↓ `analysis.py: analyze_text()` → `normalize_text()` – collapses whitespace → `segment_sentences()` – splits on punctuation → abstract proxy – first 3 sentences, max 1200 chars → TF-IDF keyword extraction – top 8 uni/bigrams ↓ Creates Paper ORM object, inserts to SQLite ↓ `semantic.py: SemanticIndex.add_paper()` → `chunk_text()` – filter boilerplate, 6-sentence windows → `sentence-transformers encode()` – 384-dim vectors → `faiss.IndexFlatIP.add()` – vectors stored in RAM → EvidenceChunk list updated ↓ Returns PaperResponse JSON to frontend ↓ Frontend: `paperFromApi()` parses metadata → Extracts real title, authors, year, venue, DOI from metadata JSON → Prepends to papers state list

11.2 Semantic Search Workflow

User types query in header search bar or chat ↓ Frontend: GET `/api/search?q={query}&top_k=6` ↓ Backend: `get_semantic_index()` – lazy singleton `ensure_index_loaded()` – loads all papers from DB if index is empty ↓ `semantic.py: SemanticIndex.search(query, top_k)` → encode query → 384-dim vector, L2-normalised → `faiss.IndexFlatIP.search(vector, k)` → map indices back to EvidenceChunk objects → return top-k results with similarity scores ↓ Frontend displays results with paper title and relevance %

11.3 RAG Chat Workflow

User types question in Chat page ↓ Frontend: POST `/api/chat {question, top_k, paper_id?}` ↓ Backend: `RAGService.answer(question, top_k, paper_id)` ↓ Step 1 – Retrieve: `SemanticIndex.search(question, top_k)` → top-k evidence chunks with similarity scores ↓ Step 2 – Filter: Drop chunks where score < 0.2 If no chunks remain → return "Insufficient evidence" (no_evidence status) ↓ Step 3 – Build prompt: `build_evidence_prompt(question, evidence)` → system instructions: answer ONLY from evidence, no invented facts → user question → all evidence chunks with [Paper ID | Title | Chunk ID | Score] headers ↓ Step 4 – Generate: `GeminiService.generate(prompt)` → Groq API → LLaMA-3.3-70B-Versatile → 1,024 max tokens,

```
temperature 0.7 ↓ Step 5 – Return: {answer, sources: [{paper_id, paper_title, chunk_id,
similarity_score, evidence}], status: "generated" | "fallback", model} ↓ Frontend renders
answer + source cards with similarity percentages
```

11.4 Intelligence Gap Detection Workflow

```
IntelligencePage mounts ↓ Frontend: GET /api/intelligence/gaps?top_k=5 ↓ intelligence.py:
detect_gaps(semantic_index, top_k) ↓ Run 3 semantic queries: "Limitations and constraints..."
"Future work and unresolved problems..." "Missing methods, weak evaluation..." ↓ For each
query: SemanticIndex.search(query, top_k * 3) ↓ Filter each result: score >= 0.3 text length >=
150 chars at least 2 sentences not already seen (deduplicate by paper_id + chunk_index) ↓ Sort
by similarity score descending Return list of gap objects {gap_type, title, evidence, paper_id,
paper_title, score} ↓ GET /api/intelligence/roadmap calls build_roadmap(gaps) → Each gap →
roadmap step with objective, methodology, metrics, phase ↓ Frontend displays gap cards with
confidence % and roadmap steps
```

12. Each Module — What It Does

Module	File	Uses	What it does
API + ORM	backend/app/main.py	FastAPI, SQLAlchemy, all modules	Entry point. Defines all REST routes. Manages SQLite database. Wires together all AI/ML services. Handles PDF upload, metadata storage, CORS.
NLP Analysis	backend/app/analysis.py	PyMuPDF, scikit-learn TF-IDF, spaCy	Extracts text and metadata from PDF bytes. Segments sentences. Extracts TF-IDF keywords. Generates abstract proxy. Runs on every upload.

Module	File	Uses	What it does
Semantic Index	<code>backend/app/semantic.py</code>	sentence-transformers, FAISS, numpy	Embeds paper text into 384-dim vectors using all-MiniLM-L6-v2. Stores in FAISS flat inner-product index. Chunks text into 6-sentence windows with boilerplate filtering. Provides vector similarity search.
RAG Pipeline	<code>backend/app/rag.py</code>	SemanticIndex, GeminiService	Orchestrates retrieve-then-read Q&A. Retrieves evidence, filters by quality threshold, builds evidence-bounded prompt, calls LLM. Returns answer + sources with status.
LLM Service	<code>backend/app/gemini.py</code>	Groq SDK, httpx	Wraps the Groq API (LLaMA-3.3-70B). Provides <code>async generate(prompt)</code> with graceful fallback on failure. Reads <code>GROQ_API_KEY</code> from env.
Research Intelligence	<code>backend/app/intelligence.py</code>	SemanticIndex (via <code>detect_gaps</code>)	Detects research gaps by semantic queries (limitations, future work, method gaps). Generates roadmap steps from gap templates. Extracts comparison metadata from paper ORM objects.

Module	File	Uses	What it does
Analytics	backend/app/analytics.py	scikit-learn KMeans + TF-IDF, pandas	Clusters papers by topic using K-Means on TF-IDF vectors with silhouette scoring. Analyses keyword frequency and ingestion trends using pandas.
Frontend App	artifacts/researchpilot/src/App.tsx	React, Wouter, Clerk, Tailwind	All 8 pages/routes. Auth routing. API communication. Client-side citation formatting. PDF metadata parsing. Global semantic search. Settings persistence.

13. API Endpoints Reference

Health

GET /healthz Response: {"status": "ok"}

Papers

GET /api/papers Headers: X-User-Email: user@example.com Response: [PaperResponse, ...] POST /api/papers Headers: X-User-Email: user@example.com Body: multipart/form-data file=<PDF> Response: PaperResponse (201 Created) GET /api/papers/{paper_id}/file Response: PDF file (inline) PATCH /api/papers/{paper_id} Headers: X-User-Email: user@example.com Body: {"title?", "collection?", "tags"?: [], "notes?", "favorite"?} Response: PaperResponse

Search

GET /api/search?q={query}&top_k={1-20}&paper_id={optional} Response: [SemanticSearchResult, ...] {paper_id, title, text, chunk_index, score}

Chat

POST /api/chat Body: {"question": "...", "top_k": 5, "paper_id": null} Response: ChatResponse {question, answer, sources: [{paper_id, paper_title, chunk_id, similarity_score, evidence}], status, model}

Intelligence

```
GET /api/intelligence/gaps?top_k=3 Response: [{gap_type, title, evidence, paper_id,
paper_title, chunk_id, similarity_score}, ...] GET /api/intelligence/roadmap?top_k=3 Response:
[{step, gap, problem, objective, methodology, metrics, phase, source}, ...] GET
/api/intelligence/comparison Response: [{paper_id, title, year, keywords, abstract,
sentence_count}, ...]
```

Analytics

```
GET /api/analytics/clusters Response: {status, cluster_count, silhouette_score, clusters:
[...], assignments: [...]} GET /api/analytics/trends Response: {status, paper_count,
ingestion_years: [...], keyword_frequency: [...]}
```

Index

```
POST /api/index/rebuild Response: {status: "ok", papers_indexed, chunks_indexed}
```

Documentation generated for ResearchPilot AI — September 2026